



COS 132 — Imperative Programming

Foundation Test #2 Memo

Worked Solutions

Module:	COS 132 — Imperative Programming
Assessment:	Foundations Test #2 (ST2 Programme)
Set by:	Swift Tuition
Total Marks:	24

Programme note. The premium COS 132 course is available at www.swifttuition.com/tuks/cos132. Learners watching before ST2 2026 may use coupon code **200COS132** at checkout for R200 off the full price. This material is Day 1 of the 7-day programme. All three foundation tests are included; one test may be taken now and the others anytime after ST2. Where units are mentioned, they refer to Swift Tuition's split of the module's lecture content into learning units covering lectures 1–16.

1 Overview

Good day, good people. Below is the worked solution for Foundations Test #2. Read the reasoning, not just the answers — the goal is for the next question of this kind to feel obvious instead of tricky.

Total marks: 24

Question 1 [8 marks]

(a) True or False

i. False. [1 mark]

Two functions with the same name can be overloaded only if the compiler can tell them apart from the **call site** — that is, from the arguments you pass in. Return type alone is not enough, because the compiler picks the overload before it even thinks about what to do with the return value.

So this is a compile error:

```
int compute(int x);  
double compute(int x);    // ERROR: same parameter list as the int version
```

But this is fine:

```
int compute(int x);  
double compute(double x);    // OK: parameter type is different
```

The parameter list is the disambiguator. The return type is not.

ii. False. [1 mark]

This is one of the most common integer-division traps. Trace it left to right:

5 / 2 — both operands are int, so this is integer division. Result is 2 (the .5 is discarded).

2 + 3.0 — now we have int + double. C++ promotes the int to double, so this is 2.0 + 3.0 = 5.0.

The whole expression evaluates to 5.0, not 5.5. The .5 was lost in the very first step. By the time the 3.0 shows up to "rescue" us, it's too late.

The lesson: integer division is decided by the *immediate* operands, not by what happens later in the expression.

iii. True. [1 mark]

This is called **fall-through**, and it's a defining feature of switch. The way to picture it: imagine the case labels as doors along a corridor. You enter through whichever door matches your value, but once inside, you keep walking forward through every door after it. The only thing that stops you is a break statement, which acts as a wall. The end of the switch block is also a wall.

So in this example:

```
switch (day) {  
    case 1: cout << "Mon ";  
    case 2: cout << "Tue ";  
    case 3: cout << "Wed ";  
    case 4: cout << "Thu "; break;  
    case 5: cout << "Fri ";  
}
```

If day = 2, you enter through the door at case 2, print Tue, keep walking, print Wed, keep walking, print Thu, then hit the wall (break) and exit. Output: Tue Wed Thu.

Most of the time, fall-through is a bug — the student wanted just Tue and forgot to put a break after each case. But sometimes it's deliberate. If you want several cases to run the same code, you stack them with no statements in between:

```
switch (grade) {  
    case 'A':  
    case 'B':  
    case 'C':  
        cout << "Pass";  
        break;  
    case 'F':  
        cout << "Fail";  
        break;  
}
```

Here, A, B and C all flow into the same cout. The doors are right next to each other with no walls between them. That's fall-through used on purpose.

Why does C++ work this way? Because it inherited the behaviour from C, and C inherited it from older languages where case labels were essentially line-labels you could jump to. The default-fall-through is a historical quirk, not a feature anyone designed deliberately. Modern languages like Rust and modern Java have cleaned this up by making each case stop automatically. C++ kept the old way for backward compatibility.

iv. True. [1 mark]

&& (AND) short-circuits. The way to picture it: imagine you're checking off a list of conditions, and someone asks you "are ALL of these true?" The moment you find one that's false, you stop checking — there's no point looking at the rest, because the answer is already "no".

C++ does exactly that. If the left operand of && is false, the whole expression is false regardless of what the right operand would say. So C++ doesn't bother evaluating the right operand at all. It just walks away.

|| (OR) is the same idea in reverse. Someone asks "is AT LEAST ONE of these true?" The moment you find one that's true, you stop — the answer is already "yes". So if the left operand of || is true, the right side is skipped.

This matters in real code, not just in exam questions, because the right side might have a side effect — calling a function, modifying a variable, doing a calculation. Short-circuit evaluation is what lets you write defensive code like:

```
if (count > 0 && total / count > 50) { ... }
```

If count is zero, the second part `total / count` would be a division by zero, which is undefined behaviour. But because of short-circuit, the second part is never evaluated when count is zero. The && walks away before the division happens, and the program is safe.

Same idea with pointers (which you'll see in Unit H):

```
if (p != NULL && p->value > 0) { ... }
```

If p is NULL, the `p->value` part would crash the program. Short-circuit means it's never evaluated. The if quietly evaluates to false and life goes on.

(b) Multiple choice

i. C. `compute(1, , 3);` is invalid. [1 mark]

C++ doesn't have "named arguments" or skipping. When you call a function, the compiler matches arguments to parameters left to right. You can stop early and let the defaults fill in the missing right-

hand arguments — but you can't skip a middle one.

So:

- `compute(1)` — passes 1 for a, defaults take care of b and c. Valid.
- `compute(1, 2)` — passes 1, 2 for a, b, default for c. Valid.
- `compute(1, , 3)` — tries to pass 1 for a, skip b, pass 3 for c. Compile error.
- `compute(1, 2, 3)` — all three explicit. Valid.

The rule: defaults can only be omitted from the right.

ii. C. 20. [1 mark]

This is a **nested ternary**. Read it from the outside in.

The outer ternary is $(x > 5) ? \text{something} : x$.

- $x > 5$ with $x = 10$ is true.
- So the result is the something branch.

The something branch is itself a ternary: $(x < 15) ? x * 2 : x / 2$.

- $x < 15$ with $x = 10$ is true.
- So this evaluates to $x * 2 = 20$.

The final result is 20.

Nested ternaries are valid C++ but they get hard to read fast. In your own code, you often want to break a nested ternary into a few `if/else` blocks for clarity. The exam might still test you on reading them.

iii. B. Skips the rest of the current iteration and proceeds to the next. [1 mark]

`continue` jumps to the loop's update step (in a `for`) or back to the condition check (in a `while`). The rest of the loop body — anything below the `continue` — is skipped for *that* iteration only.

The other options:

- **A** describes `break`, not `continue`.
- **C** doesn't exist as a C++ keyword.
- **D** is just wrong — `continue` is fully legal inside a `for` loop.

iv. C. do-while. [1 mark]

This is the defining feature of `do-while`: the body runs first, *then* the condition is checked. So even if the condition is false from the start, the body has already executed once.

Compare:

- `for` and `while` check the condition **before** the first iteration. If it's false, the body never runs.
- `do-while` checks the condition **after** each iteration. The first iteration always happens.

This is why `do-while` is the natural fit for input validation patterns: ask the user for input, then check if the input is valid; if not, ask again.

Question 2 [8 marks]

(a) Output: 25 20 [2 marks]

This question is testing **scope**. There are two variables called `x` floating around:

- The **global** `x = 10`, declared at file scope.
- The **local** `x = 20`, declared inside `main`.

Inside `main`, the local `x` **shadows** (hides) the global one. So when `main` says `f(x)`, it's passing the local `x`, which is 20.

Inside `f`, the parameter is *a/so* named `x`. But it's a third, separate `x` — local to the function `f`. C++ passes by value here (no reference, no pointer), so a copy of 20 is given to `f`.

`f` modifies its own local copy: `x = x + 5` makes its local `x` equal to 25. Then it prints 25 (with the trailing space).

Back in `main`, the local `x` is untouched. It's still 20. So `cout << x` prints 20.

Output: 25 20.

If you wrote 25 25, you assumed the modification inside `f` reached back to `main`. It can't, because parameters were passed by value — a copy was made. The original is untouched.

If you wrote 15 20, you used the global `x = 10` instead of the local `x = 20` inside `main`. The local always wins when both are in scope.

(b) Output: 9 [2 marks]

Trace the loop with `n = 5`:

i	i % 2 == 0?	action
1	false (1 % 2 = 1)	sum += 1 → sum = 1
2	true	continue — skip the rest
3	false (3 % 2 = 1)	sum += 3 → sum = 4
4	true	continue
5	false (5 % 2 = 1)	sum += 5 → sum = 9

So `sum` accumulates only the odd values: `1 + 3 + 5 = 9`.

If you got 15, you missed the `continue` and added everything. If you got 6, you inverted the condition and added the evens (`2 + 4`).

(c) Output: Wed Thu [2 marks]

`day` is 3, so the switch jumps into case 3: It prints `Wed`, but there's no `break` — so it falls through into case 4: That prints `Thu`, and case 4 *does* have a `break`, which exits the switch.

Cases 1, 2 and 5 are never visited at all (cases 1 and 2 are above the entry point, and case 5 is reached only if 4 didn't break).

Output: `Wed Thu` (with a trailing space, because each `cout` includes a space after the day name).

If you wrote just `Wed`, you forgot fall-through.

If you wrote `Wed Thu Fri`, you forgot that case 4 has a break.

(d) Output: 0 4 [2 marks]

This question stacks two ideas: **short-circuit evaluation** and **post-increment** in the same expression. Trace it carefully, starting with `a = 3`:

1. `a > 0` evaluates to `3 > 0 → true`. `&&` keeps going.
2. `a++ < 5` — this is two things at once:
 - Read the current value of `a` (which is 3) for the comparison: `3 < 5 → true`.
 - **After** the comparison, increment `a`. Now `a = 4`.
 - `&&` keeps going.
3. `a > 4` evaluates to `4 > 4 → false`. The whole `&&` chain is now false.

result becomes false. When you print a bool with `cout`, true shows as 1 and false shows as 0. So we print 0.

Then `cout << a` prints the current value of `a`, which is 4.

Output: 0 4.

The classic mistake: forgetting that `a++` reads the current value before incrementing. If you used the post-increment value instead, you'd get `a > 4` evaluating to `5 > 4 = true`, and the wrong answer.

Question 3 [4 marks]

Expected answer

#	Type	Line	Explanation
1	Syntax	5	Missing semicolon at the end of <code>int total = 0</code> .
2	Syntax	7	The for-loop header uses commas where it needs semicolons. The three sections (init, condition, update) must be separated by <code>;</code> not <code>,</code> .
3	Syntax	10	Missing <code><<</code> between <code>"Sum: "</code> and <code>total</code> . The line should read <code>cout << "Sum: " << total << endl;</code> .

How to think about it

When scanning code for errors, work line by line and ask three questions of each line:

1. **Does it end where it should end?** Most C++ statements end in `;`. If a line ends without one, that's almost always a syntax error.
2. **Does the punctuation match what the keyword expects?** A for loop expects `;` separators. A function call expects `,` between arguments. Don't confuse them.
3. **Is every `<<` or `>>` between two adjacent things?** A break in the chain (like `cout << "Sum: " total`) means a missing operator.

All three errors here are **syntax** errors — they break the grammar of the language, so the compiler refuses to compile. None of them are semantic (which would mean the code compiles but does the wrong thing).

Marks breakdown

- 1 mark per error correctly identified (with type, line and a sensible explanation).
- 1 bonus mark if all three are caught with no false alarms.

Common pitfalls

- Misreading these as **semantic** errors. They're not — they're all violations of grammar, so the compiler can't even build the program.
- Spotting only two of the three. The missing `<<` on line 10 is easy to miss because line 5 and line 7 are more visually obvious.

Question 4 [4 marks]

Expected answer (most natural)

```
void countdown(int start) {  
    for (int i = start; i >= 1; i--) {  
        cout << i << endl;  
    }  
}
```

How to think about it

The Logo procedure does this:

1. Takes one parameter (:START).
2. Sets up a counter :N that starts at :START.
3. Repeats :START times, printing :N and decrementing.

You don't actually need the auxiliary :N in C++ — the loop counter itself can do the job. Just count *down* instead of up. Start at `start` and decrement until you reach 1.

If you prefer to mirror the Logo source line-for-line, the literal translation also works:

```
void countdown(int start) {  
    int n = start;  
    for (int i = 0; i < start; i++) {  
        cout << n << endl;  
        n = n - 1;  
    }  
}
```

This is fully correct and earns full marks. It's just a longer way of saying the same thing.

Marks breakdown

- 1 mark: correct function signature `void countdown(int start)`.
- 1 mark: the loop runs the right number of times (`start` iterations).
- 1 mark: each iteration prints the current value on its own line.
- 1 mark: overall correctness — counts from `start` down to 1, no off-by-one.

Other valid answers

- while-loop versions are fine. Same output, full marks.
- `cout << n << "\n";` instead of `endl` — equivalent.
- Using `--n` or `n--` instead of `n = n - 1` — same effect.

Common pitfalls

- **Off-by-one.** Printing start down to 0 instead of down to 1 is a classic. The Logo source repeats :START times, not :START + 1 times.
- **Counting up instead of down.** The output should be 5, 4, 3, 2, 1, not 1, 2, 3, 4, 5. Read the Logo source carefully — it decrements :N after each print.
- **Forgetting the newline.** The Logo PRINT command outputs each value on its own line. Use endl or "\n".
- **Returning int or some other type.** The Logo procedure returns nothing, so the C++ version is void.

After this test

This was the medium-difficulty foundations test. Roughly:

- **20–24 marks:** comfortable across all ST1 territory. Good to push into Unit G.
- **15–19 marks:** mostly solid, but specific concepts (overloading rules, scope, switch fall-through, short-circuit + side-effects) need another look.
- **10–14 marks:** real gaps — go back to the relevant Unit C, D, E or F lecture videos before pushing on.
- **Below 10:** retake Foundations #1 first; come back to this one when you're scoring above 18 there.

Logo coverage

This test contained one Logo→C++ conversion (Q4) — a REPEAT loop with an explicit counter variable. Per the COS 132 lecturer's confirmation that ST2 can ask Logo→C++ conversions including loops, this is the medium difficulty of what you might see.